

20 questions Power BI

Le test DAX /20



Pour futurs Data Analysts. Durée conseillée : 25 min · 3 paliers · corrigé détaillé.

Objectif : mesurer si tu raisones en contexte de filtre, en relations et en mesures DAX, pas seulement si tu sais cliquer les bons boutons Power BI. Ce test vérifie si tu sais pourquoi une colonne calculée fige un ratio, quand une relation inactive s'applique vraiment et pourquoi un total peut devenir faux dans une colonne calculée.

Repères : 9/20 = boutons Power BI ok, DAX fragile. 14/20 = prêt pour un entretien technique junior. 18/20 = raisonnement DAX solide.

Candidat _____ Date _____ Note _____ /20

Observation terrain : en entretien, je ne demande jamais si tu connais les slicers ou VLOOKUP. Je te montre une mesure DAX qui donne un chiffre bizarre et je regarde si tu sais dire pourquoi. C'est le seul signal qui compte.

Palier 1 : les fondations du raisonnement DAX.

Sept questions pour vérifier que tu distingues mesure et colonne calculée, contexte de ligne et contexte de filtre.

Question 1. Une mesure DAX se distingue d'une colonne calculée principalement parce que :

- a) Elle se recalcule à chaque interaction, selon le contexte de filtre affiché à l'écran.
- b) Elle est stockée physiquement dans le modèle, comme une colonne classique.
- c) Elle ne peut jamais utiliser SUM, AVERAGE ou COUNT.
- d) Elle ne s'actualise qu'au moment du refresh planifié du modèle.

Question 2. Tu crées une colonne calculée % Marge = DIVIDE(Ventes[Marge], Ventes[CA]) dans la table Ventes, ligne par ligne. Ton manager te dit que ce pourcentage affiché par région sera faux. Pourquoi ?

- a) DIVIDE ne fonctionne pas dans une colonne calculée.
- b) Il manque un ALL() autour de la division.
- c) Un ratio dépend du niveau d'agrégation affiché : c'est une mesure qu'il fallait écrire, pas une colonne calculée figée par ligne.
- d) Les colonnes calculées ne supportent pas les références entre deux colonnes de la même table.

Question 3. Que retourne CALCULATE(SUM(Ventes[Montant]), FILTER(Ventes, Ventes[Montant] > 100)) ?

- a) La somme de tous les montants de la table, sans qu'aucun filtre sur Montant ne soit appliqué à l'agrégation finale.
- b) La somme des montants des lignes où Montant est supérieur à 100.
- c) Une erreur de compilation, car FILTER n'a pas le droit d'apparaître à l'intérieur d'un CALCULATE en DAX.
- d) La liste des lignes qui respectent la condition Montant > 100, retournée sous forme de table, sans aucune agrégation appliquée.

Question 4. Ton collègue ajoute une colonne calculée `Ecart = Ventes[CA] - [CA Moyen]`, où `[CA Moyen]` est une mesure existante. Sur chaque ligne, `Ecart` affiche une valeur proche de zéro, quel que soit le produit. Où est l'erreur ?

- a) Le nom de la mesure doit être entre guillemets dans une colonne calculée, sinon DAX ne sait pas distinguer une mesure d'une colonne du même nom.
- b) Il manque un `ALL()` explicite autour de `[CA Moyen]` pour forcer le calcul sur la table entière plutôt que sur la ligne.
- c) DAX interdit la soustraction entre une colonne physique et une mesure, les deux types de valeurs n'étant pas compatibles dans une même expression.
- d) Référencer une mesure dans une colonne calculée déclenche une transition de contexte : `[CA Moyen]` s'évalue avec un filtre réduit à la ligne courante, donc il retourne le CA de cette ligne, pas la moyenne globale.

Question 5. Le contexte de ligne existe :

- a) Dans les colonnes calculées et dans les fonctions itératives comme `SUMX` ou `FILTER`, où chaque ligne est évaluée une à une.
- b) Uniquement dans les mesures qui utilisent `CALCULATE`.
- c) Uniquement dans les visuels Power BI, jamais dans les formules du modèle.
- d) Dans toutes les formules DAX, sans exception.

Question 6. Dans `SUMX(Ventes, CALCULATE(SUM(Ventes[Quantite])))`, à l'intérieur de l'itérateur `SUMX`, que fait `CALCULATE` ?

- a) Rien de particulier, `CALCULATE` est ignoré à l'intérieur d'un itérateur.
- b) Il annule le contexte de ligne et recalcule `SUM` sur toute la table, sans filtre.
- c) Il transforme le contexte de ligne courant en contexte de filtre équivalent : c'est la transition de contexte.
- d) Il provoque systématiquement une référence circulaire.

Question 7. Ton modèle a deux relations entre Ventes et Calendrier : une active sur `DateCommande`, une inactive sur `DateLivraison`. Tu veux une mesure de CA basée sur la date de livraison. Que dois-tu écrire ?

- a) Rien, la relation inactive s'active automatiquement dès qu'on utilise la colonne `DateLivraison` dans un visuel.
- b) `CALCULATE(SUM(Ventes[CA]), USERELATIONSHIP(Ventes[DateLivraison], Calendrier[Date]))` pour activer ponctuellement la relation inactive dans cette mesure.
- c) Supprimer la relation active pour forcer Power BI à utiliser l'autre.
- d) Créer une troisième table de dates dédiée à la livraison.

Palier 2 : le contexte de filtre en action.

Sept questions sur `CALCULATE`, `ALL()`, les relations et leur direction de filtrage.

Question 8. Quel est le rôle central de `CALCULATE` en DAX ?

- a) Créer une nouvelle table temporaire pour l'itération.
- b) Trier les résultats selon un critère personnalisé.
- c) Convertir une colonne en mesure.
- d) Modifier le contexte de filtre courant, en ajoutant, remplaçant ou retirant des filtres avant d'évaluer une expression.

Question 9. Le visuel est déjà filtré par un slicer sur `Region = Sud`. Dans ce contexte, `CALCULATE(SUM(Ventes[Montant]), Ventes[Region] = "Nord")` retourne :

- a) La somme des montants pour la région Nord uniquement.
- b) La somme des montants combinant les régions Nord et Sud, comme si le `CALCULATE` ajoutait son filtre à celui déjà présent dans le visuel.
- c) Zéro, parce que le filtre du slicer sur Sud et celui du `CALCULATE` sur Nord s'annuleraient mutuellement en s'excluant l'un l'autre.
- d) Une erreur de calcul, DAX refusant d'évaluer une expression dès que deux filtres contradictoires portent sur la même colonne.

Question 10. En entretien, on te demande une mesure « part du total » qui calcule la part de CA de chaque produit sur le total général, indépendamment des filtres du visuel. Quelle fonction retire les filtres pour obtenir ce total général ?

- a) `FILTER(Ventes, TRUE())`, qui renverrait toute la table sans retirer aucun filtre du contexte existant.
- b) `HASONEVALUE(Ventes[Produit])`, une fonction qui teste seulement si une seule valeur est visible, sans retirer aucun filtre.
- c) `ALL(Ventes)` ou `ALL(Ventes[Produit])`, utilisé à l'intérieur d'un `CALCULATE`.
- d) `DISTINCT(Ventes[Produit])`, qui retourne la liste des produits uniques sans modifier le contexte de filtre.

Question 11. Pour ne garder que les ventes de la région Nord, deux collègues proposent (1)

`CALCULATE(SUM(Ventes[Montant]), Ventes[Region] = "Nord")` et (2) `CALCULATE(SUM(Ventes[Montant]), FILTER(ALL(Ventes[Region]), Ventes[Region] = "Nord"))`. Pour ce cas simple, quel est le meilleur choix ?

- a) La version (2), car `FILTER(ALL())` est obligatoire pour tout filtre booléen en DAX.
- b) La version (1) : le filtre booléen simple suffit ici, il est plus court et plus lisible qu'un `FILTER(ALL())` équivalent qui n'apporte rien de plus pour ce cas.
- c) Les deux sont interdites, il faut un `KEEPFILTERS`.
- d) Aucune des deux, DAX ne permet pas de filtrer une colonne texte avec un signe égal.

Question 12. Dans une relation one-to-many entre Calendrier (un) et Ventes (plusieurs), le filtre se propage :

- a) De Ventes vers Calendrier, jamais l'inverse.
- b) Dans les deux sens automatiquement, sans configuration.
- c) Uniquement si on ajoute `CALCULATE` dans la mesure.
- d) Du côté « un » (Calendrier) vers le côté « plusieurs » (Ventes) : sélectionner une date filtre les ventes correspondantes.

Question 13. Ta mesure CA Livraison doit utiliser la relation inactive entre Ventes[DateLivraison] et Calendrier[Date] au lieu de la relation active sur DateCommande. Si tu oublies `USERELATIONSHIP` dans le `CALCULATE`, que se passe-t-il ?

- a) La mesure utilise silencieusement la relation active sur DateCommande, au lieu de la relation inactive sur DateLivraison.
- b) Power BI refuse de compiler la mesure et affiche une erreur de syntaxe tant que `USERELATIONSHIP` n'est pas ajouté au `CALCULATE`.
- c) Les deux relations, celle sur DateCommande et celle sur DateLivraison, s'activent en même temps automatiquement dès que la mesure est évaluée.
- d) La mesure retourne systématiquement un résultat vide, faute de relation active reconnue par le moteur DAX pour cette mesure.

Question 14. Un collègue propose de rendre bidirectionnelle la relation entre Ventes et Calendrier « pour que ça marche dans tous les sens ». Quel est le principal risque ?

- a) Power BI limite à trois relations bidirectionnelles par modèle.
- b) Les relations bidirectionnelles ralentissent uniquement l'actualisation, jamais les calculs.
- c) Le filtrage croisé dans les deux sens peut créer de l'ambiguïté ou des boucles de filtre entre plusieurs tables, avec des résultats de mesure imprévisibles.
- d) Une relation bidirectionnelle transforme automatiquement toutes les colonnes calculées en mesures.

Palier 3 : les pièges de pro.

Six questions de time intelligence et de transition de contexte cachée, là où se joue le 18/20.

Question 15. La mesure `TOTALYTD(SUM(Ventes[CA]), Calendrier[Date])` fonctionne correctement seulement si :

- a) La table Calendrier contient uniquement les jours ouvrés.
- b) La colonne Calendrier[Date] appartient à une table de dates marquée comme table de dates dans le modèle, avec une plage continue sans trous.
- c) La mesure `SUM(Ventes[CA])` est remplacée par `AVERAGE`.
- d) Le modèle ne contient qu'une seule table.

Question 16. Un collègue calcule le YoY (année sur année) avec

`SAMEPERIODLASTYEAR(Ventes[DateCommande])`, en utilisant directement la colonne de date de la table de faits, sans table de calendrier séparée. Les chiffres semblent incohérents d'un mois à l'autre. Quelle est l'erreur ?

- a) `SAMEPERIODLASTYEAR` n'existe pas en DAX.
- b) Il fallait utiliser `SUMX` à la place de `SAMEPERIODLASTYEAR`.
- c) Le problème vient uniquement du fuseau horaire des dates.
- d) Sans table de calendrier dédiée, continue et marquée, les fonctions de time intelligence n'ont pas de base fiable pour se décaler d'un an, ce qui produit des résultats faux ou incohérents.

Question 17. On te demande une mesure de cumul depuis le début de l'année, qui se met à jour à chaque nouveau mois filtré. Quelle fonction correspond exactement à ce besoin ?

- a) `TOTALYTD(SUM(Ventes[CA]), Calendrier[Date])` ou l'équivalent `CALCULATE(SUM(Ventes[CA]), DATESYTD(Calendrier[Date]))`, qui cumulent depuis le 1^{er} janvier du contexte jusqu'à la dernière date filtrée.
- b) `SAMEPERIODLASTYEAR(Calendrier[Date])` utilisé seul, sans l'envelopper dans un `CALCULATE`, en pensant que cela suffit à décaler la date.
- c) `SUM(Ventes[CA])` filtré manuellement par un slicer d'année, en demandant à l'utilisateur de changer l'année lui-même chaque mois.
- d) `ALLTIME(Ventes[CA])`, une fonction qui n'existe pas dans le langage DAX.

Question 18. Tu veux le CA filtré sur la région Nord ET sur l'année 2025 (les deux conditions vraies en même temps). Quelle écriture est correcte ?

- a) `CALCULATE(SUM(Ventes[CA]), Ventes[Region] = "Nord" || Calendrier[Annee] = 2025)`
- b) `CALCULATE(SUM(Ventes[CA]), FILTER(Ventes, Ventes[Region] = "Nord") || FILTER(Calendrier, Calendrier[Annee] = 2025))`
- c) `CALCULATE(SUM(Ventes[CA]), Ventes[Region] = "Nord", Calendrier[Annee] = 2025)`, deux arguments de filtre séparés dans `CALCULATE` se combinent automatiquement en ET.
- d) `CALCULATE(SUM(Ventes[CA]), OR(Ventes[Region] = "Nord", Calendrier[Annee] = 2025))`

Question 19. Sur une carte Power BI, tu poses simplement le champ Ventes[Montant], sans écrire de mesure : Power BI agrège automatiquement en somme. Pourquoi une mesure explicite reste préférable en production ?

- a) Une agrégation implicite est toujours plus rapide qu'une mesure explicite, car Power BI évite l'étape de compilation du moteur DAX.
- b) Une mesure explicite est nommée et réutilisable partout, avec un comportement cohérent, contrairement à l'agrégation implicite qui dépend du visuel.
- c) Power BI interdit les agrégations implicites depuis 2023, une mesure explicite étant devenue obligatoire pour tout champ numérique posé.
- d) Une agrégation implicite ne peut jamais être une somme, elle se limite au comptage et à la moyenne selon le type de la colonne.

Question 20. Un collègue écrit une colonne calculée `CA Filtre Nord = CALCULATE(SUM(Ventes[CA]), Ventes[Region] = "Nord")` directement dans la table Ventes. Le total affiché dans un visuel semble faux. Quelle est la cause ?

- a) `CALCULATE` est interdit dans une colonne calculée : Power BI refuse de compiler la formule et affiche une erreur de syntaxe.
- b) `SUM` ne peut pas être utilisé à l'intérieur d'un `CALCULATE`, il faudrait le remplacer par `SUMX` pour que la formule fonctionne.
- c) Il manque un `ALL()` pour retirer le filtre de région avant l'agrégation, sans quoi `CALCULATE` ignore la condition posée.
- d) Dans une colonne calculée, `CALCULATE` déclenche une transition de contexte ligne par ligne, ce qui fausse le total attendu.

Solutions et explications.

Un point par bonne réponse. Compte ton score, puis relis surtout celles que tu ne peux pas expliquer à voix haute.

- 1. a)** La mesure ne stocke rien, elle se réévalue au contexte de filtre du visuel. Le piège classique : confondre le recalcul au refresh, propre à une colonne calculée, avec le comportement réel d'une mesure.
- 2. c)** Une colonne calculée fige un ratio au grain de la ligne source; sommer ces pourcentages par région ne redonne jamais le vrai ratio agrégé. Le piège classique : croire qu'une colonne calculée suit le filtre du visuel comme une mesure.
- 3. b)** CALCULATE transforme la table retournée par FILTER en contexte de filtre pour SUM. Le piège classique : penser que FILTER renvoie directement un nombre, alors qu'il renvoie une table que CALCULATE convertit en filtre.
- 4. d)** Une mesure référencée dans une colonne calculée subit une transition de contexte : elle s'évalue avec un CALCULATE implicite qui restreint le filtre à la ligne courante, donc [CA Moyen] renvoie le CA de cette ligne, pas la moyenne de la table, ce qui ramène Ecart proche de zéro partout. Le piège classique : croire que la mesure garde son comportement « table entière » alors qu'elle hérite du contexte de ligne, exactement comme dans la question 20.
- 5. a)** Le contexte de ligne apparaît partout où DAX parcourt une table ligne par ligne : colonnes calculées et itérateurs. Le piège classique : croire qu'une simple mesure agrégée a aussi un contexte de ligne, alors qu'elle démarre sans lui.
- 6. c)** C'est la transition de contexte : dès qu'un CALCULATE apparaît dans un contexte de ligne, DAX convertit cette ligne en filtre. Le piège classique : oublier que cette transition change le résultat par rapport à une simple SUM sans CALCULATE.
- 7. b)** USERELATIONSHIP active une relation inactive uniquement pour la durée d'évaluation de la mesure, sans toucher au reste du modèle. Le piège classique : croire qu'une relation inactive s'active automatiquement selon les colonnes utilisées dans le visuel.
- 8. d)** CALCULATE est la fonction pivot de DAX : elle réécrit le contexte de filtre avant de recalculer l'expression qu'elle contient. Le piège classique : la réduire à une fonction pour ajouter un filtre, alors qu'elle peut aussi en retirer.
- 9. a)** Un filtre écrit dans CALCULATE sur une colonne déjà filtrée remplace ce filtre, il ne s'additionne pas au filtre existant du slicer. Le piège classique : croire que le filtre du slicer et celui du CALCULATE se cumulent en ET.
- 10. c)** ALL() retire les filtres existants sur la table ou la colonne visée, ce qui permet de calculer un total de référence pour un ratio de part de total. Le piège classique : utiliser DISTINCT ou FILTER, qui ne retirent aucun filtre, ils listent ou re-filtrent seulement.
- 11. b)** Quand le filtre est un simple test d'égalité, la syntaxe booléenne directe fait exactement la même chose que FILTER(ALL()), sans complexité additionnelle. Le piège classique : généraliser une bonne pratique valable pour des filtres complexes à tous les cas, y compris les plus simples où elle n'apporte rien.
- 12. d)** Par défaut, une relation one-to-many filtre dans le sens un vers plusieurs : la table qui porte la clé unique contrôle le filtre appliqué à la table de faits. Le piège classique : inverser le sens et croire que filtrer la table de faits change automatiquement la table de dimension.
- 13. a)** Sans USERELATIONSHIP, DAX ignore silencieusement la relation inactive et applique la relation active par défaut, sans message d'erreur. Le piège classique : croire qu'un résultat affiché sans erreur est forcément correct.
- 14. c)** Activer le filtrage dans les deux sens ouvre la porte à des chemins de filtre multiples entre tables, ce qui peut produire des résultats ambigus dès que le modèle grandit. Le piège classique : activer le bidirectionnel par sécurité, sans mesurer l'effet sur les autres relations.
- 15. b)** Les fonctions de time intelligence s'appuient sur une vraie table de dates continue, marquée comme telle dans le modèle : sans ça, les cumuls deviennent silencieusement faux. Le piège classique : utiliser la colonne de date de la table de faits directement, sans table de calendrier dédiée.
- 16. d)** Les fonctions de time intelligence exigent une table de dates dédiée, marquée, avec une plage continue ; utiliser une colonne de date de la table de faits, souvent trouée ou dupliquée, casse le calcul. Le piège classique : penser que n'importe quelle colonne de date suffit tant qu'elle est au format date.
- 17. a)** DATESYTD et TOTALYTD cumulent le CA depuis le début de l'année civile du contexte jusqu'à la dernière date visible. Le piège classique : confondre cumul depuis le début d'année avec même période l'an dernier, qui répond à une autre question.
- 18. c)** Deux filtres passés comme arguments distincts dans CALCULATE se combinent toujours en ET logique; pour un OU explicite, il faut construire une seule table de filtre avec FILTER et l'opérateur ||. Le piège classique : utiliser || directement entre deux arguments de CALCULATE, une syntaxe qui n'exprime pas ce qu'on croit.
- 19. b)** Une mesure explicite porte un nom stable, une définition unique, réutilisable dans tous les visuels; une agrégation implicite dépend du champ posé et du réglage local du visuel. Le piège classique : croire qu'un chiffre juste sur un visuel isolé prouve la fiabilité du modèle.
- 20. d)** Ecrire CALCULATE dans une colonne calculée déclenche une transition de contexte pour chaque ligne individuellement, très différent d'une mesure évaluée une seule fois au niveau du visuel. Le piège classique : copier un réflexe de mesure dans une colonne calculée sans anticiper l'effet ligne par ligne.

Observation terrain : le vrai signal recruteur n'est pas la note affichée, c'est ta capacité à expliquer pourquoi une réponse fausse est fausse.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

